

Memory Efficient Doubly Linked List

–Delete At Position

–Program Analysis and Time Complexity

Program:

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    // Traverse to the node at 'pos'
    while (curr != nullptr && i < pos)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }
```

```

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    // We are deleting 'curr'.
    // Logic: Connect 'prev' directly to 'next'.

    next = XOR(prev, curr->npx); // Decode node after curr

    // 1. Update prev
    // prev->npx was (prevPrev ^ curr). Needs to be
    (prevPrev ^ next).
    Node *prevPrev = XOR(prev->npx, curr);
    prev->npx = XOR(prevPrev, next);

    // 2. Update next (if exists)
    // next->npx was (curr ^ nextNext). Needs to be (prev ^
    nextNext).
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(prev, nextNext);
    }

    cout << "Deleted " << curr->data << " from position " <<
pos << "." << endl;
    free(curr);
}

.....

    case 7:
        cout << "Enter position to delete: ";
        cin >> pos;
        deleteFromPosition(pos);
        break;

```

Program Analysis

1. When List Is Empty

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

Node * head

Address

Head

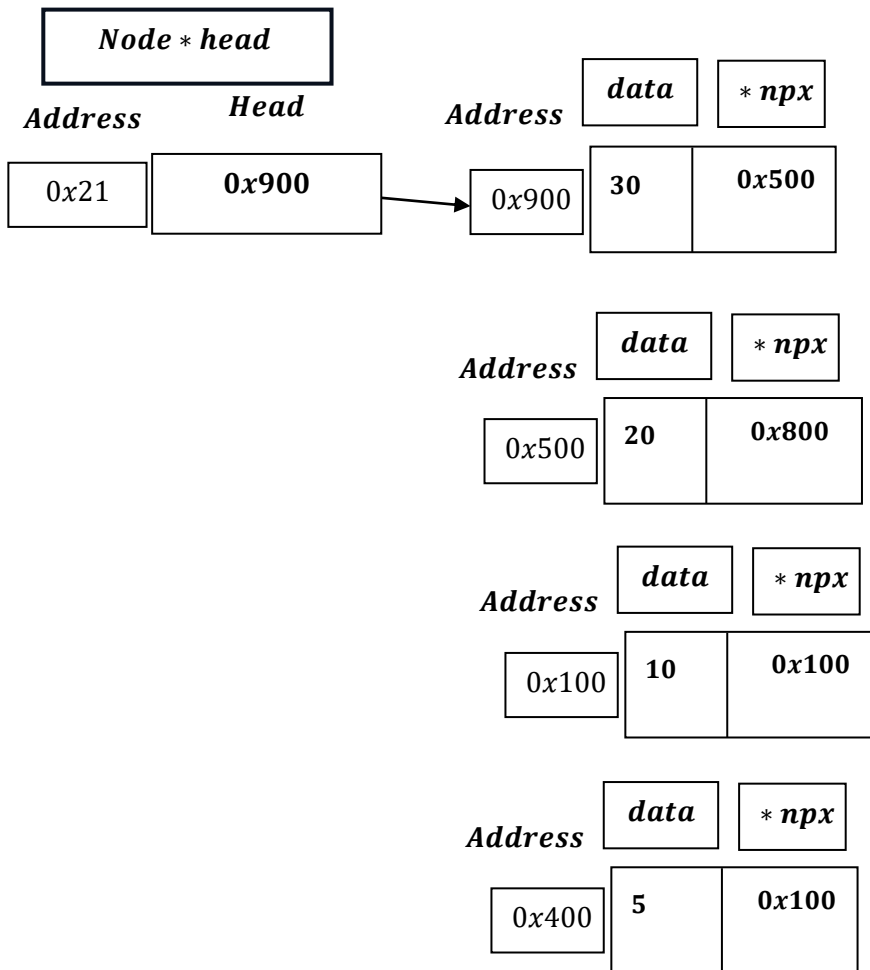
0x21

nullptr

Then , it prints : "List is empty.Nothing to delete."

Return Void and Exit.

2. When List Is Not Empty



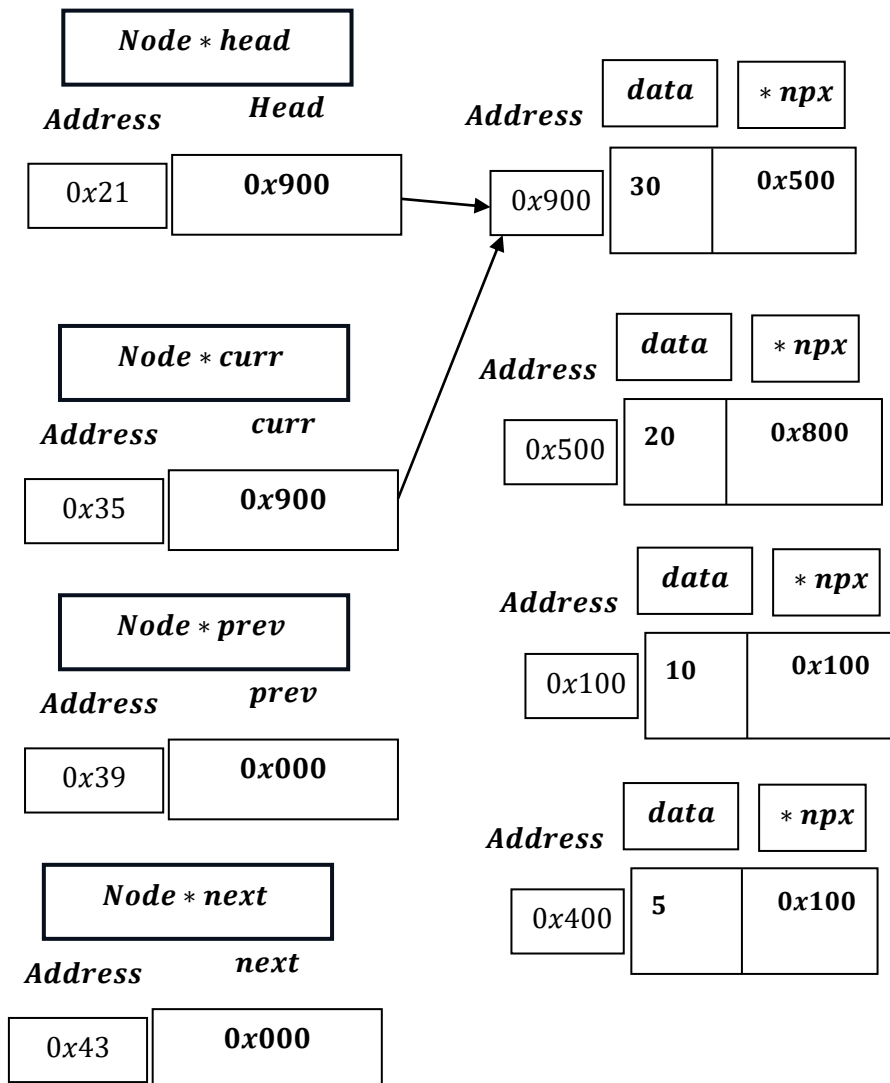
```
cout << "Enter position to delete: ";  
cin >> pos;  
deleteFromPosition(pos);  
break;
```

if pos = 1, then:

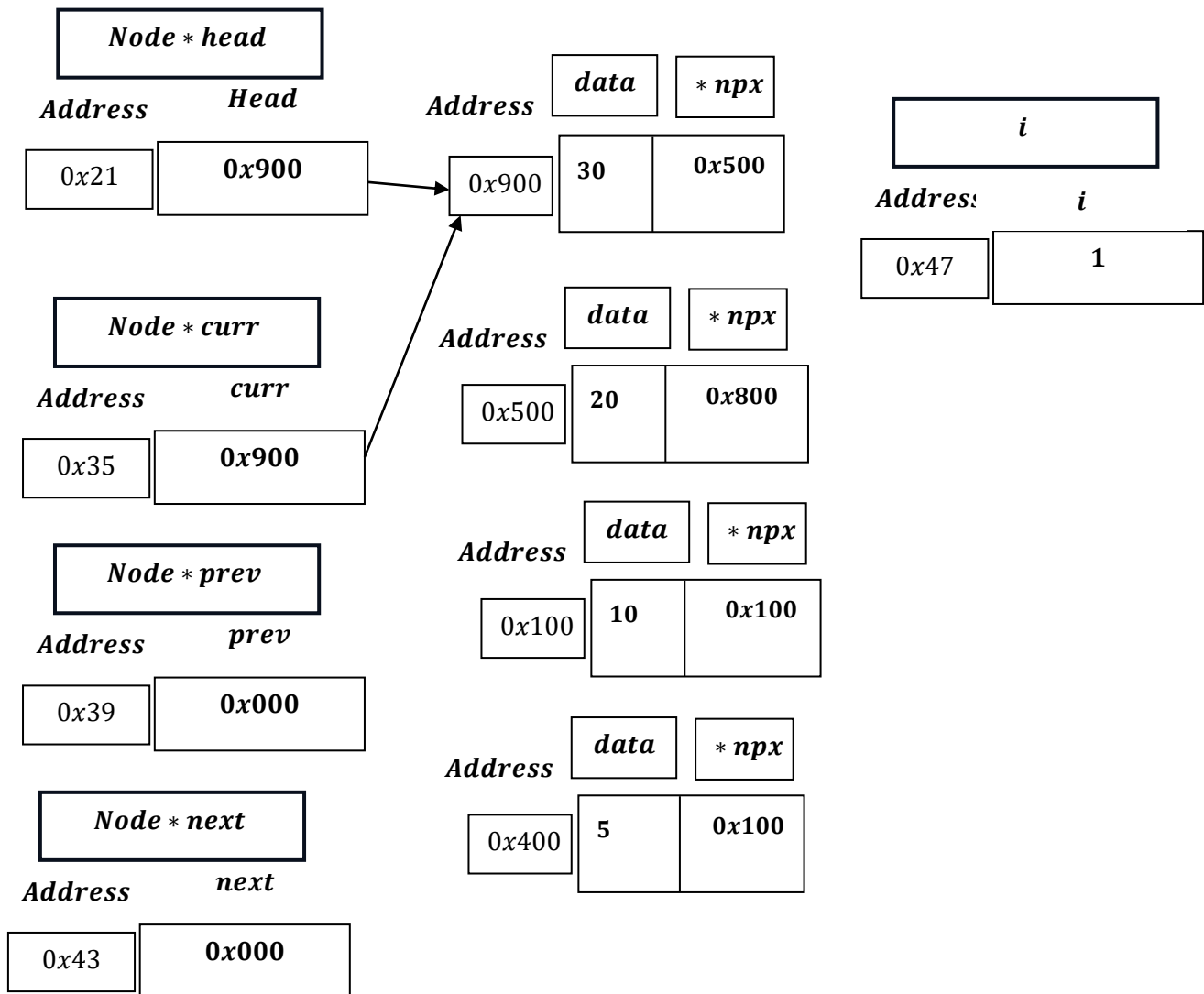
```
if (pos == 1)
{
    deleteFromBeginning();
    return;
}
```

Then it runs deleteFromBeginning() and return statement exit from the function.

```
Node *curr = head;
Node *prev = nullptr;
Node *next;
```



```
int i = 1;
```



```
while (curr != nullptr && i < pos)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

Lets say pos: 3.

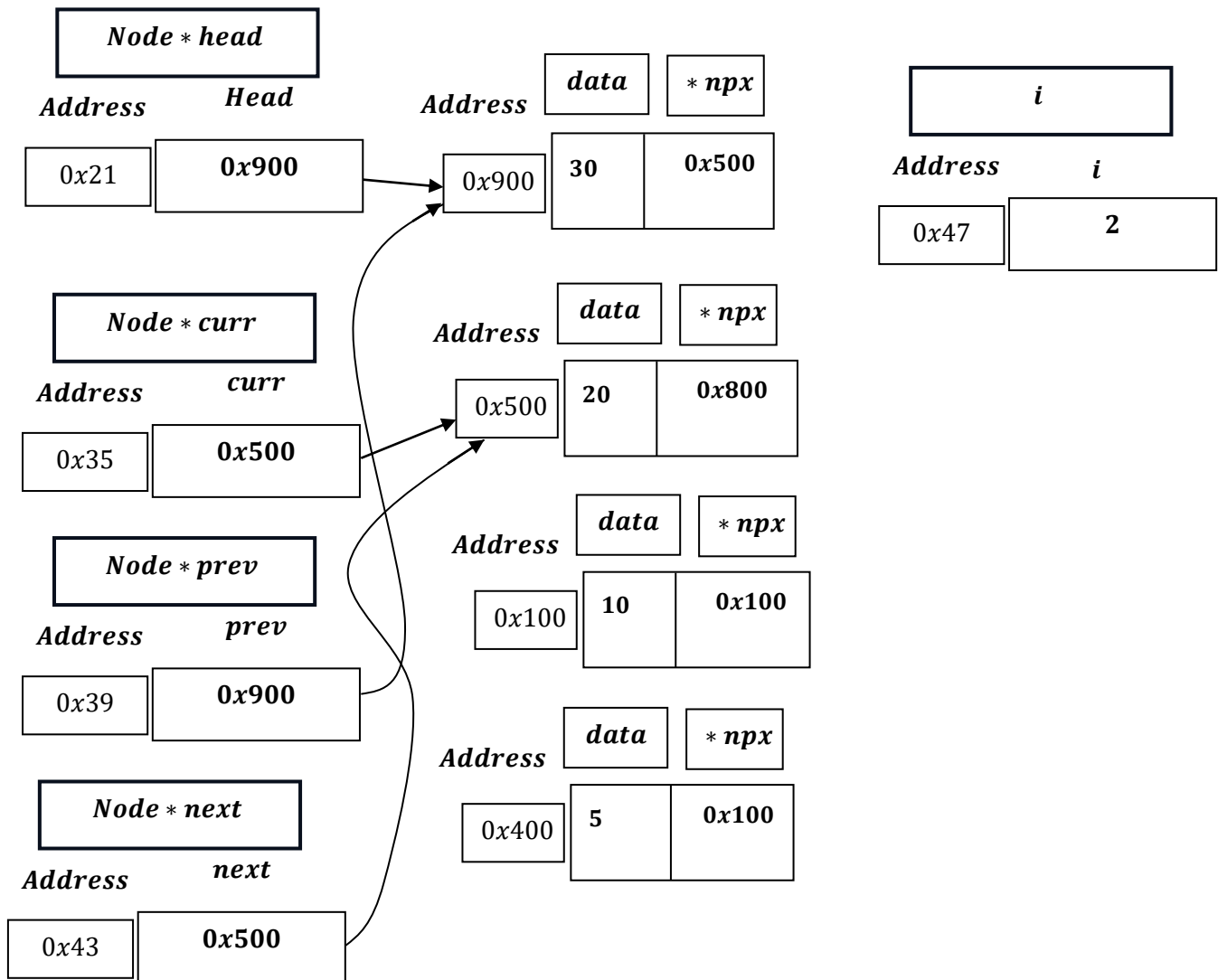
***curr: 0x900* $\neq$ *nullptr: 0x000* AND *i = 1* < *pos: 3* :**

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x000 $\oplus$ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

i = i + 1 = 1 + 1 = 2.



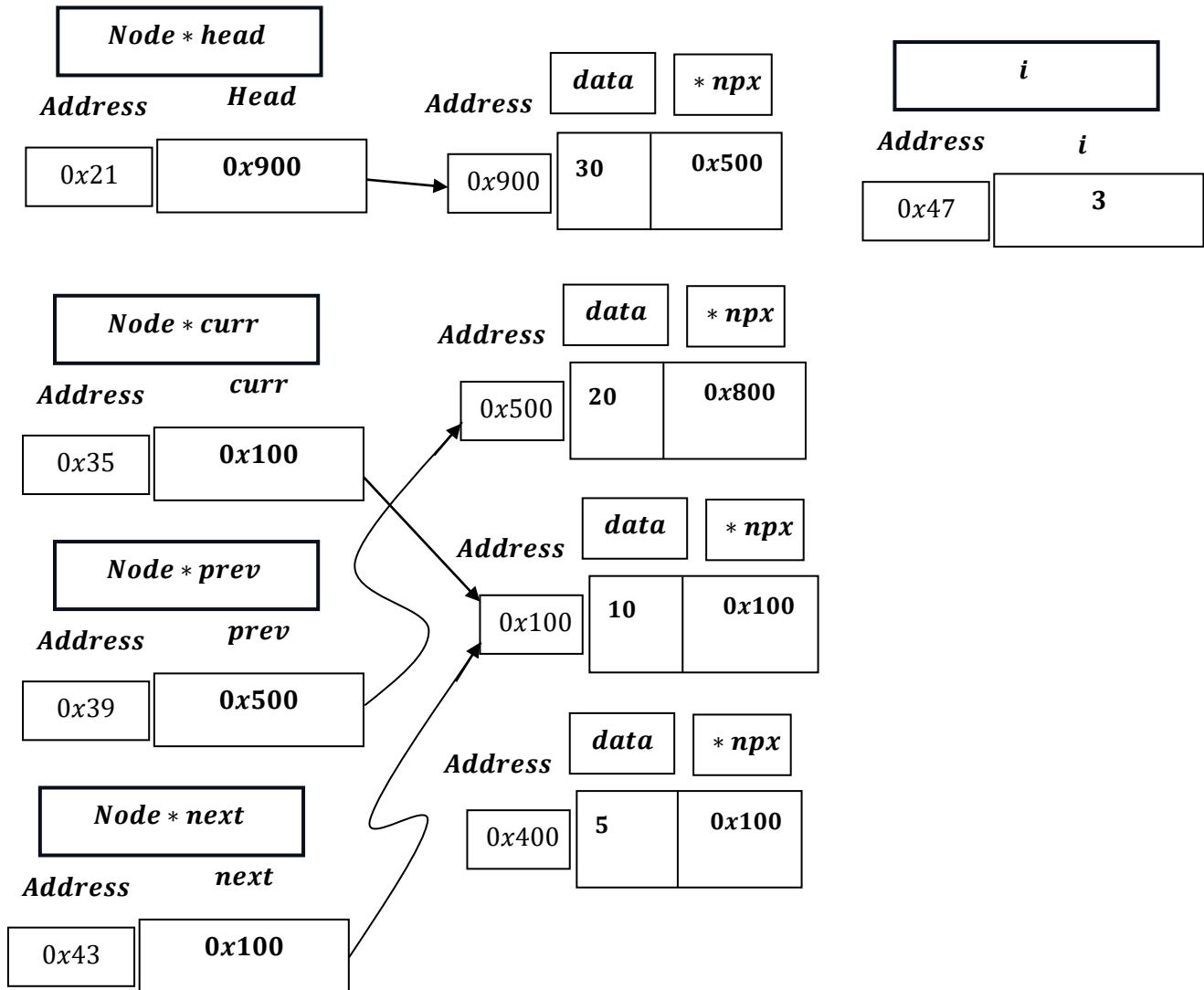
curr: 0x500 $\neq$ nullptr: 0x000 AND $i = 2 < pos: 3$:

next = XOR(prev: 0x900, curr $\rightarrow$ npx: 0x800) = $1001_2 \oplus 1000_2 = 0001_2 = 0x100$

prev = curr = 0x500

curr = next = 0x100

$i = i + 1 = 2 + 1 = 3$.

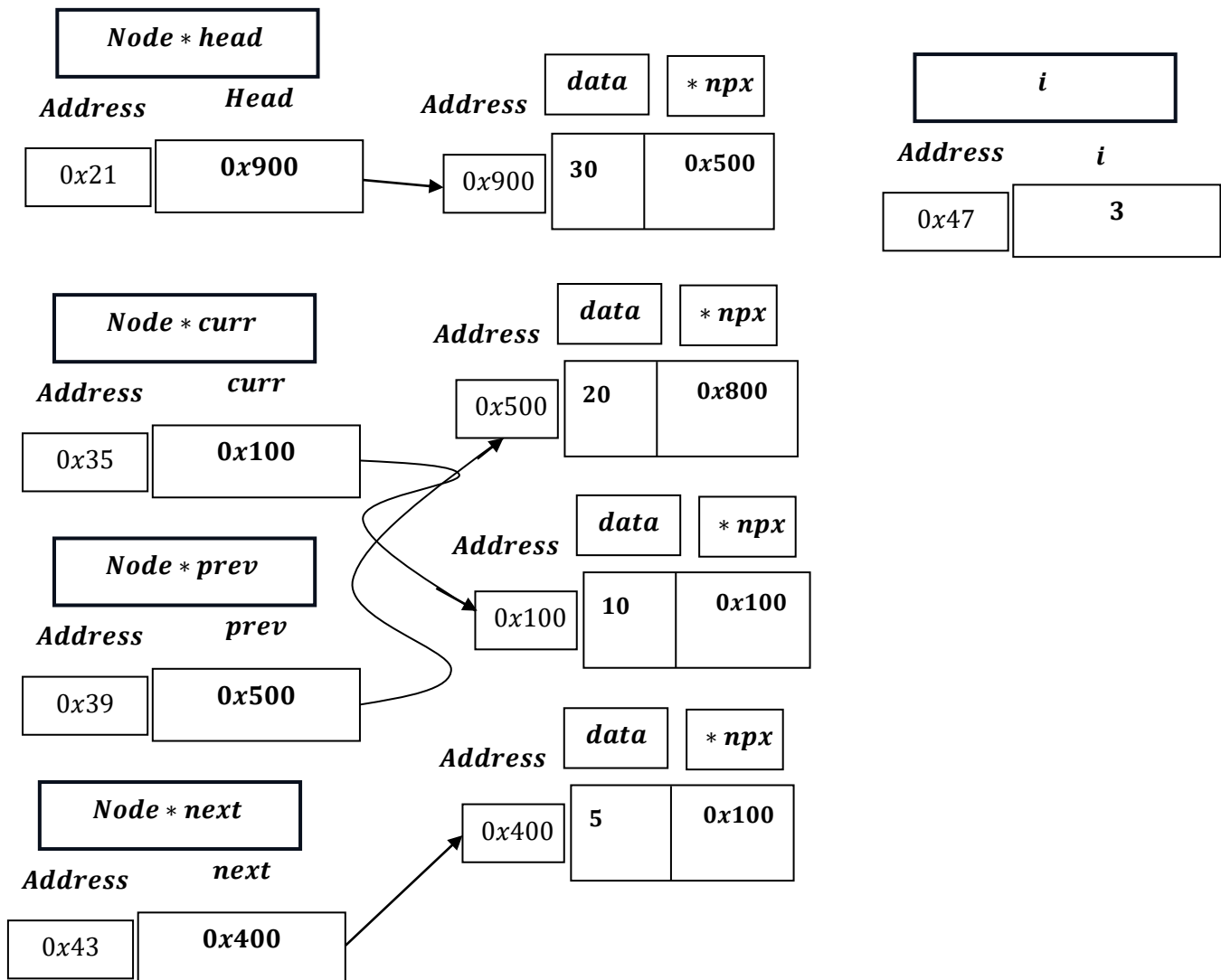


***curr: 0x100 $\neq$ nullptr: 0x000 [is TRUE] AND $i = 3 < pos: 3$ [is False]
 $\Rightarrow$ Condition is False.***

```
next = XOR(prev, curr->npx);
```

$next = XOR(prev: 0x500 \oplus curr \rightarrow npx: 0x100);$

$0101_2 \oplus 0001_2 = 0100_2 = 4_{10} = 0x400.$



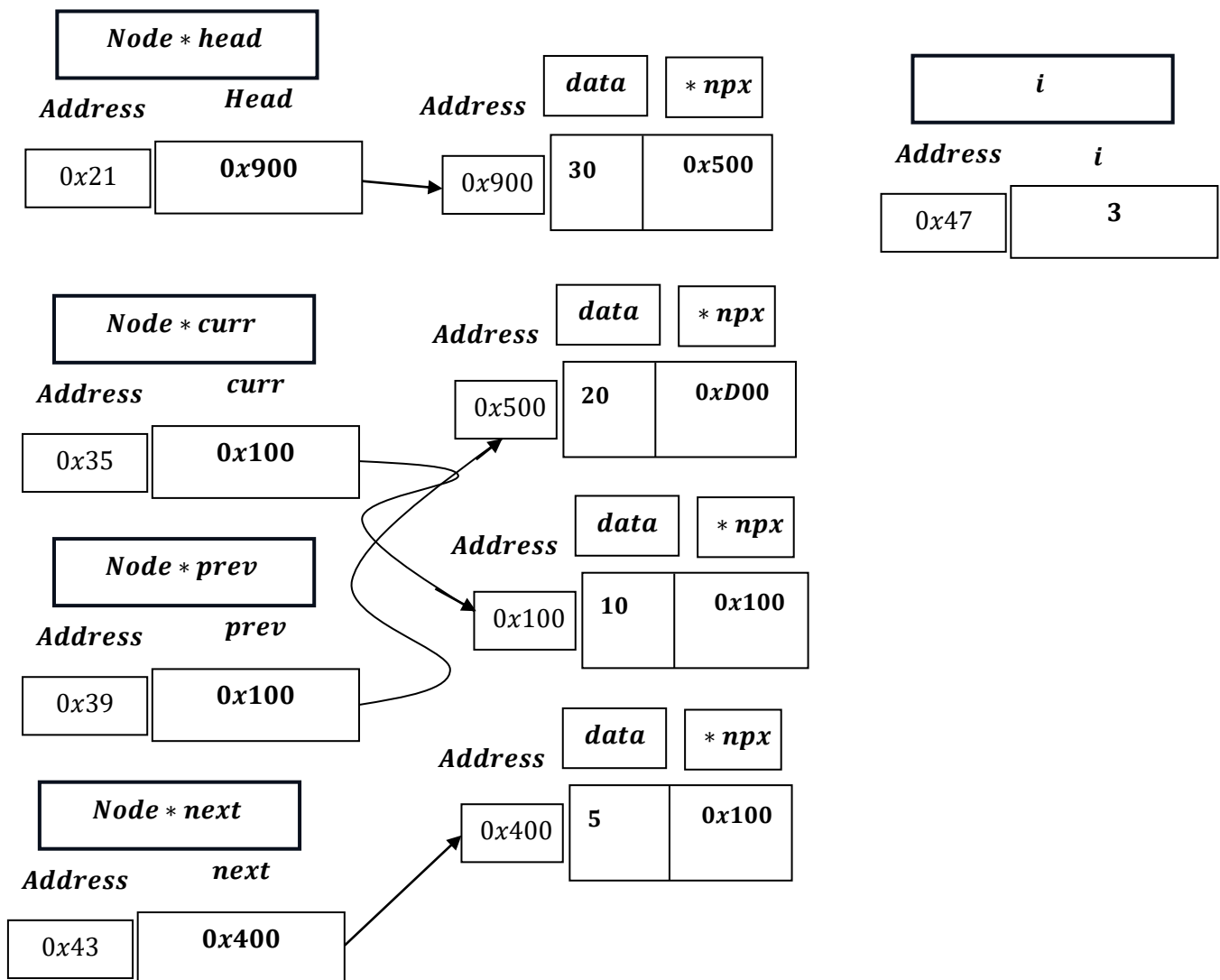
```
Node *prevPrev = XOR(prev->npx, curr);
prev->npx = XOR(prevPrev, next);
```

$prevPrev = XOR(prev \rightarrow npx: 0x800, curr: 0x100)$

$= 1000_2 \oplus 0001_2 = 1001_2 = 9_{10} = 0x900$

$prev \rightarrow npx = XOR(prevPrev: 0x900, next: 0x400)$

$= 1001_2 \oplus 0100_2 = 1101_2 = 13_{10} = 0xD00$



```

if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(prev, nextNext);
}

```

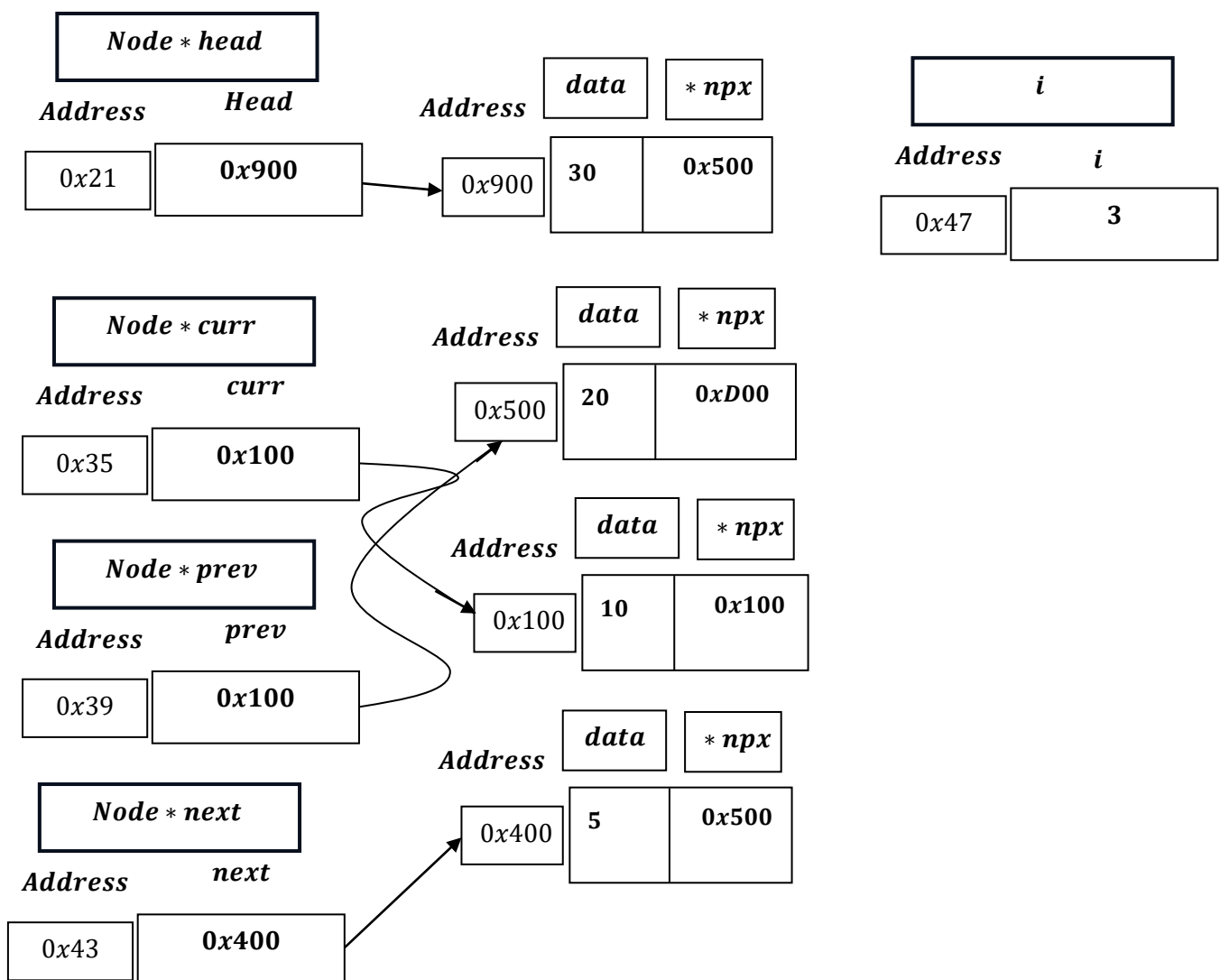
***next: 0x400* $\neq$ *nullptr: 0x000* :**

***nextNext* = *XOR*(*curr: 0x100*, *next* $\rightarrow$ *npx: 0x100*)**

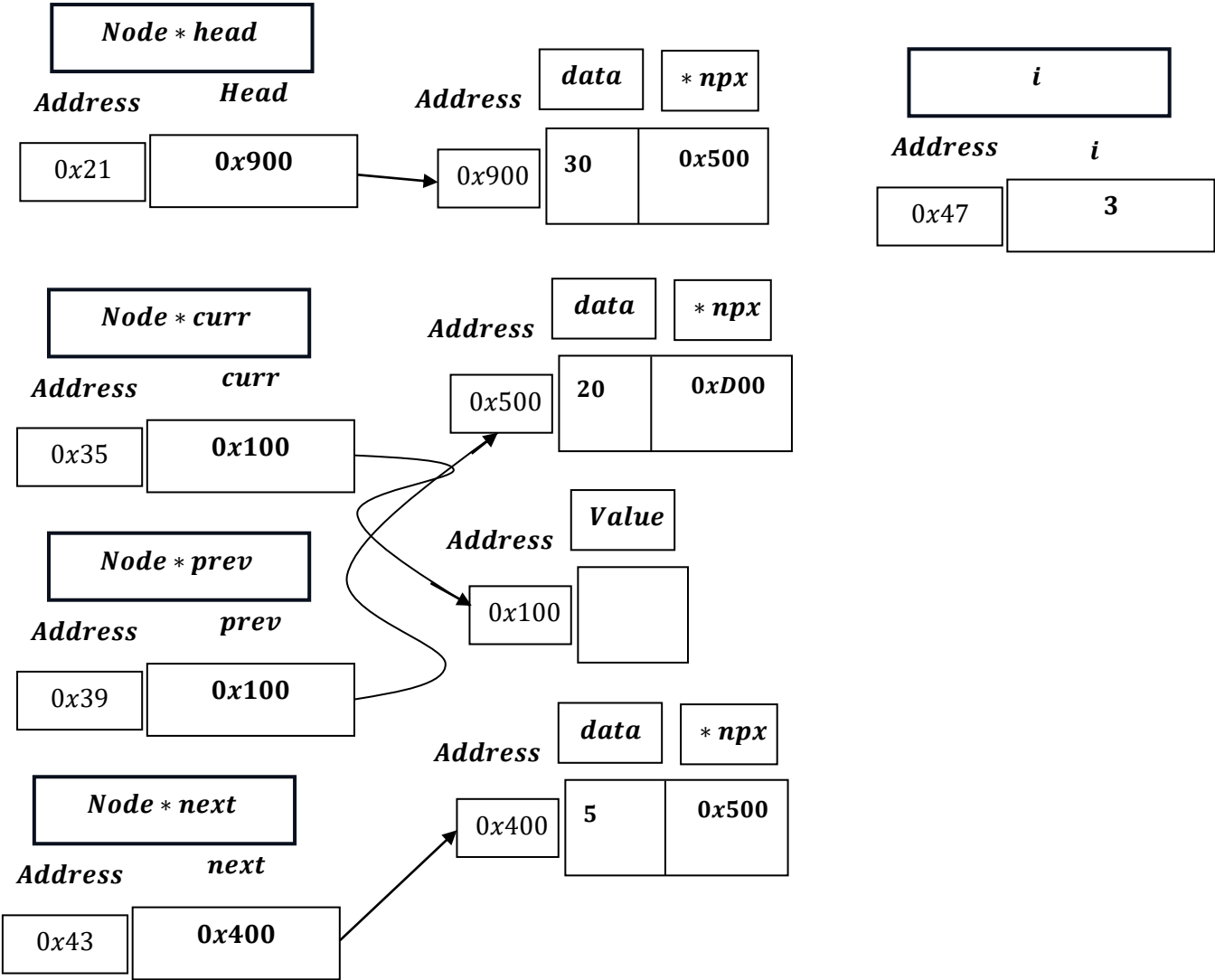
$= 0001_2 \oplus 0001_2 = 0000_2 = 0x000$.

***next* $\rightarrow$ *npx* = *XOR*(*prev: 0x500*, *nextNext: 0x000*)**

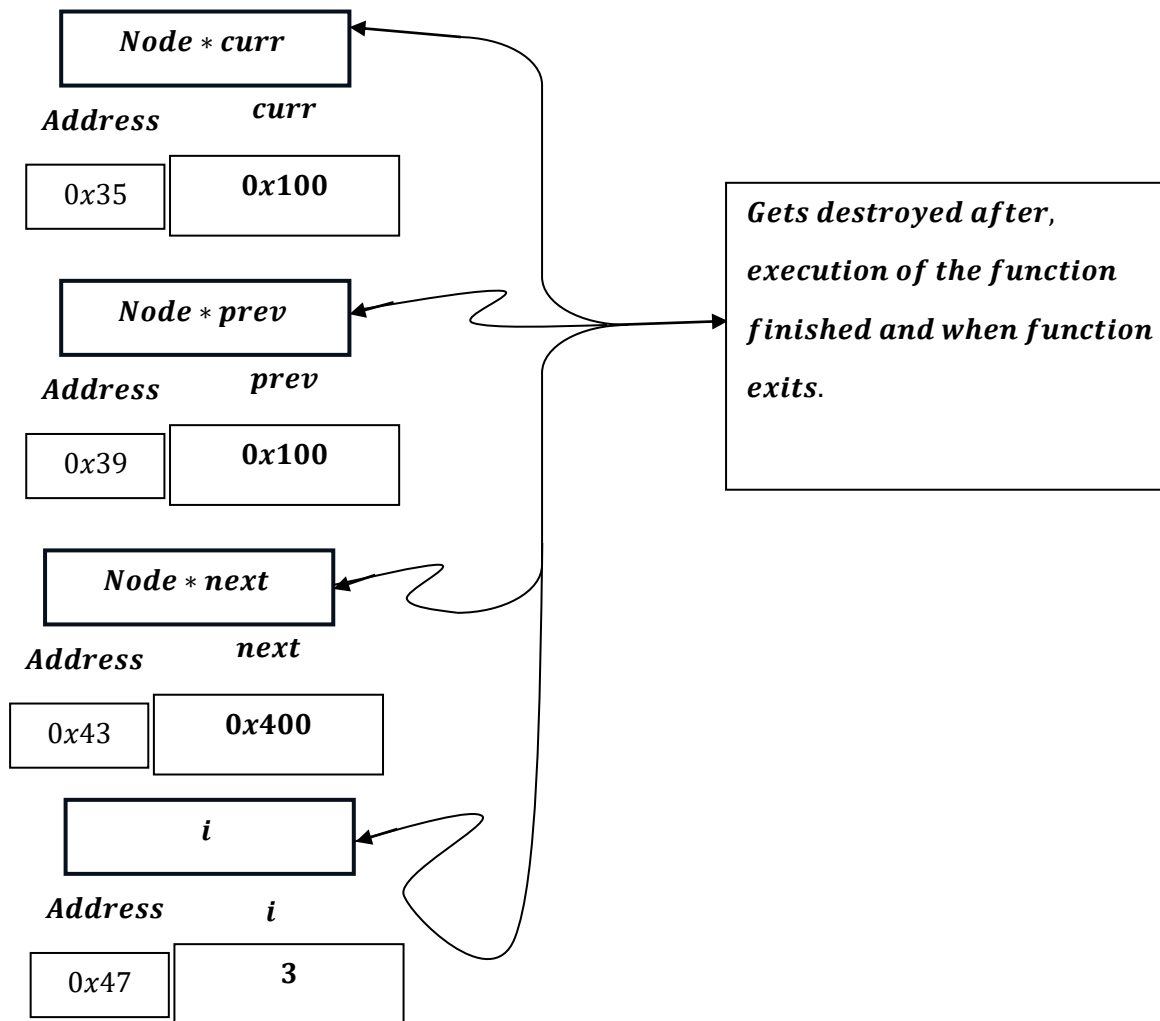
$= 0101_2 \oplus 0000_2 = 0101_2 = 5_{10} = 0x500$



```
free(curr);
```



As , functions ends:



And we are left with:

